DATAROBOT · EXPLAINABILITY APPS · DEPLOYMENT GUIDE

Deploying the Generic Explainability App to DataRobot Codespaces

How to configure and launch the app in a DataRobot environment using a public GitHub repository.

PREREQUISITES

- DataRobot API token (from **Developer Tools → API Keys**) — only needed when running outside a Codespace; inside a Codespace the token is injected automatically
- A DataRobot deployment with SHAP prediction explanations enabled (preferred), or a project + model ID
- An Data Registry dataset ID to score — use `create_scoring_sample.py` to generate one if needed

01

Clone the repository

Once your Codespace is open, clone the repository into persistent storage and navigate into the app subfolder.

```
git clone https://github.com/olga-shpyrko-dr/explainability-apps.git ~/storage/explainability-apps
cd ~/storage/explainability-apps/generic-explainability
```

Keep `.env` out of version control — it is already in `.gitignore`. The domain config files (`narrative_config.json`, `profile_config.json`, `feature_group_mapping.json`) are safe to edit and commit for a specific deployment.

02

Configure parameters

Gather these values before running anything. How you supply them depends on the deployment path:

- **Local Codespace testing (steps 03–04):** copy `backend/.env.template` to `backend/.env` and fill in the values.
- **Custom Application deployment (steps 05–07):** run `dr dotenv setup && dr dotenv validate` — the CLI stores secrets in DataRobot's secret management. No `.env` file needed.

PARAMETER	DESCRIPTION	
DATAROBOT_API_TOKEN	Auto-injected in a Codespace — only set manually when running outside one	AUTO
DATAROBOT_ENDPOINT	`https://app.eu.datarobot.com/api/v2` — adjust region	REQUIRED

PARAMETER	DESCRIPTION	
DEPLOYMENT_ID	Deployment with SHAP enabled — preferred scoring path	RECOMMENDED
PROJECT_ID + MODEL_ID	Fallback if no deployment is available. Set only one path.	FALLBACK
SCORING_DATASET_ID	Data Registry dataset ID to score on startup. If you don't have one yet, run the helper script to generate and upload a sample: `cd backend && python create_scoring_sample.py --upload --dataset-name "Scoring Sample" --use-case-id <DEFAULT_USE_CASE_ID>` The script prints the new dataset ID to use here.	REQUIRED
ROW_ID_COL	Unique row identifier column name (e.g. `CLAIM_ID`)	REQUIRED
OUTCOME_COL	Binary 0/1 outcome column — enables actual outcome rate in narrative	OPTIONAL
DEFAULT_USE_CASE_ID	Scopes the in-app dataset selector to one use case folder	OPTIONAL
DR_GATEWAY_MODEL	LLM Gateway model for AI narrative — check **Registry → LLM Gateway** for available model names, e.g. `azure/gpt-4o-mini`	OPTIONAL
APP_TITLE / APP_SUBTITLE	Displayed in the app header	OPTIONAL

03

Customise domain configuration

Edit the three JSON files in `backend/` to match your use case. Commit them back to the repo (or a deployment-specific fork) so the configuration is reproducible.

`feature_group_mapping.json`	Maps model feature names to named business domain groups shown in the Group Explanations chart. All explanation feature names must appear here — unmatched names fall into "Other".
`profile_config.json`	Which columns appear in the cohort filter panel and individual profile view, plus score and explanation filter defaults.
`narrative_config.json`	Entity labels, score labels, recommended action hint, and optional custom LLM prompt overrides.

04

Launch and test locally in the Codespace

Run everything with the provided helper script from the `generic-explainability/` directory:

```
cd ~/storage/explainability-apps/generic-explainability
chmod +x start-codespace.sh build-app.sh
./start-codespace.sh
```

The script installs Python dependencies, builds the React frontend, and starts the FastAPI backend. This is a local test run inside the Codespace — the app is not yet deployed as a persistent Custom Application.

> **Expose ports before starting the session.** Ports can only be added when the Codespace session is not running. Before launching the script, stop the session, go to **Session environment → + Add port** in the Codespace sidebar, add the frontend port (`8080`) and optionally the backend port (`5173`), then start a new session. DataRobot will provide a shareable URL for each exposed port.

> **First load takes 1–2 minutes** while the app runs a batch prediction job against `SCORING_DATASET_ID`. The page will show a loading indicator until results are ready. Subsequent loads use the local cache and start instantly — the cache persists for the lifetime of the Codespace session. To pre-warm the cache before sharing the URL with others, open the app yourself first and wait for it to finish loading.

05

Scaffold the App Framework base

The repo already has `start-app.sh` , `backend/` , and `frontend/` . The AF copier wizard adds the infrastructure manifest needed for `dr run deploy` without overwriting existing files.

```
cd ~/storage/explainability-apps/generic-explainability
uvx copier copy https://github.com/datarobot-community/af-component-base .
```

Answer the wizard prompts as follows:

PROMPT	VALUE
`Recipe name`	**`explainability-app`** — this becomes a prefix of your stack name.
`Stack name`	Your deployment identifier, e.g. `DRDemo_AI_Explainability_<UseCase>`
`All other prompts`	Accept defaults (press Enter)

> The final Pulumi stack name is `explainability-app-<your-stack-name>`. This is visible in `dr run deploy` output and in the DataRobot Applications list.

06

Build the frontend and deploy

The Custom Application container has no Node.js. Build the React frontend locally before deploying — the container serves the pre-compiled static assets directly.

```
dr dotenv setup     # prompted once; stores secrets in DR secret management
dr dotenv validate
./build-app.sh
dr run deploy
```

Pulumi creates the Custom Application, uploads the bundle, and prints the shareable URL when complete. The `start-app.sh` entry point installs Python dependencies and starts the FastAPI backend on port 8080.

> On first load the app runs a batch prediction job — this takes approximately 1–2 minutes. The page shows a loading indicator until results are ready. Subsequent loads use cached results.

07

Redeploy after changes

```
cd ~/storage/explainability-apps/generic-explainability
git pull
./build-app.sh          # only needed if frontend files changed
dr run deploy
```

To tear down the Custom Application:

```
dr run infra:down
```

OPERATIONAL NOTES

→ The batch prediction cache (`backend/.batch_output_cache.csv` and `backend/.prediction_dataset_cache.json`) is local to the Codespace session. It is not bundled into the Custom Application container — the deployed app always runs a fresh batch job on first load. To clear the cache for a new local test run, delete both files and restart the script.

→ Set only one scoring path: either `DEPLOYMENT_ID` (preferred) or `PROJECT_ID + MODEL_ID` — not both. `DEPLOYMENT_ID` takes priority when both are set.

→ All feature names that appear in `EXPLANATION_N_FEATURE_NAME` columns must be present in `feature_group_mapping.json`. Unmatched names are grouped under "Other" and will not appear in the Group Explanations chart by group. Download a completed batch job result from the DR Console → Batch Jobs to inspect actual feature names.

→ To switch the active dataset mid-session without restarting, use the dataset selector in the app's cohort panel — it calls `POST /api/dataset/switch` on the backend.